



SEDE
SANTO DOMINGO

***UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE
SEDE SANTO DOMINGO DE LOS TSÁCHILAS***



***DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS
CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN***

PERIODO : PREGRADO S-I ABR 25 - AGO 25

ASIGNATURA : Programación Orientada a Objetos

TEMA : C++ y GITHUB

ESTUDIANTE : Sebastian Rodriguez, Ángel Rodriguez, Corina Acosta, Mishelle Nuñez

NIVEL-PARALELO – NRC : Segundo B NRC: 23069

DOCENTE : Ing. Jhon Cruz

FECHA DE ENTREGA : 30/04/2025

SANTO DOMINGO – ECUADOR

Contenido

I. Introducción.....	3
II. 2. Objetivos	3
II.1. Objetivo General:	3
II.2. Objetivos Específicos:.....	3
III. 3. Desarrollo / Marco Teórico/ Práctica.....	4
III.1. Marco Teórico	4
III.2. Desarrollo.....	5
III.3. Práctica.....	6
IV. 4. Conclusiones	7
V. 5. Recomendaciones	8
VI. 6. Anexos:	9
VII. 7. Bibliografía/ Referencias	11

I. Introducción

La programación es el proceso mediante el cual se diseñan, escriben y mantienen instrucciones que una computadora puede interpretar y ejecutar para realizar tareas específicas. Estas instrucciones se redactan utilizando lenguajes de programación, los cuales permiten comunicar al hardware lo que debe hacer para resolver problemas, automatizar tareas o interactuar con los usuarios. La programación no solo es una herramienta técnica, sino también un proceso lógico y creativo que involucra el análisis de problemas, la construcción de algoritmos y la implementación de soluciones eficientes. En un mundo cada vez más digitalizado, aprender a programar se ha convertido en una habilidad esencial no solo para ingenieros y desarrolladores, sino para cualquier profesional que busque comprender o construir soluciones tecnológicas. A través de la programación se han desarrollado desde simples aplicaciones hasta complejos sistemas que transforman la manera en que vivimos y trabajamos.

II. 2. Objetivos

II.1. Objetivo General:

Adquirir conocimientos fundamentales de programación que permitan comprender conceptos básicos, fortalecer el pensamiento lógico y aplicar soluciones prácticas a problemas computacionales de manera estructurada y eficiente.

II.2. Objetivos Específicos:

- Comprender los conceptos fundamentales de la programación, como algoritmos, variables, operadores y estructuras de control, para comprender su función en la resolución de problemas computacionales.

- Explicar la importancia del pensamiento lógico y estructurado en el diseño de algoritmos eficientes, como base para el desarrollo de programas funcionales y organizados.
- Aplicar principios básicos de la programación mediante ejemplos prácticos que permitan al estudiante desarrollar habilidades para escribir, probar y depurar código.

III. 3. Desarrollo / Marco Teórico/ Práctica

III.1. Marco Teórico

La programación estructurada es una metodología basada en dividir un programa en bloques o módulos, cada uno encargado de una tarea específica. Este enfoque permite desarrollar software más claro, legible y fácil de mantener. En el lenguaje C++, la programación modular se implementa separando el código en archivos .h (cabecera) y .cpp (implementación), siguiendo el principio de responsabilidad única.

Una de las buenas prácticas más importantes en programas interactivos es la validación de entradas. Cuando un usuario introduce datos por teclado, pueden ocurrir errores (como ingresar letras en lugar de números). Para evitar fallos en tiempo de ejecución, se emplea el manejo de excepciones, que permite detectar y controlar estas situaciones de forma segura mediante bloques try-catch.

Además, el uso de estructuras de control como if, switch, for y while facilita la lógica condicional y repetitiva del programa. A través de estas estructuras, es posible implementar menús interactivos, cálculos, procesamiento de vectores y otros tipos de lógica que dependen de decisiones del usuario.

III.2. Desarrollo

El programa fue diseñado de forma modular, separado en distintos archivos que cumplen funciones específicas:

- **utils.h / utils.cpp:** Contienen funciones auxiliares que permiten leer valores numéricos del usuario con seguridad, validando que sean datos correctos mediante conversiones con `std::stoi` y `std::stod`.
- **menu.h / menu.cpp:** Se encarga de mostrar el menú de opciones, capturar la selección del usuario y redirigir a las funciones correspondientes.

Las funcionalidades implementadas son:

- **Saludo:** Muestra un mensaje amigable de bienvenida.
- **Suma y comparación:** Permite ingresar dos números, sumarlos y determinar cuál es mayor, menor o si son iguales.
- **Estadísticas:** Solicita al usuario varios números, los almacena en un vector y calcula el valor máximo, el mínimo y el promedio.
- **FizzBuzz:** Recorre un rango de números e imprime 'Fizz', 'Buzz' o 'FizzBuzz' según su divisibilidad. Se presta especial atención al manejo de errores. Cuando el usuario introduce datos no válidos (como texto en lugar de números), el programa no se bloquea ni lanza errores visibles, sino que guía al usuario con un mensaje para repetir la entrada. Programa Modular en C++ con un menú Interactivo

III.3. Práctica

Durante la ejecución del programa, el usuario interactúa con un menú principal donde puede elegir distintas opciones. Cada funcionalidad está acompañada de mensajes que guían paso a paso lo que debe realizar. Este enfoque interactivo y educativo permite aplicar de forma práctica conceptos como:

- *Lectura de datos y validación*
- *Uso de vectores dinámicos (`std::vector`)*
- *Aplicación de ciclos (`for`, `while`)*
- *Implementación modular*
- *Manejo de excepciones con `try-catch`*

Además, la ejecución del algoritmo FizzBuzz demuestra cómo se puede aplicar lógica condicional combinada en ciclos para generar salidas automáticas según patrones definidos.

La experiencia práctica permite reforzar los conocimientos teóricos sobre programación estructurada, modularidad y control de errores, al tiempo que demuestra cómo estos principios se integran en una solución concreta, interactiva y robusta.

El desarrollo y prueba del programa se realizó en el entorno de desarrollo integrado (IDE) Code::Blocks, utilizando su compilador predeterminado para C++. Este entorno facilitó la organización de archivos, la depuración y la ejecución continua del programa de manera eficiente y amigable para el usuario.

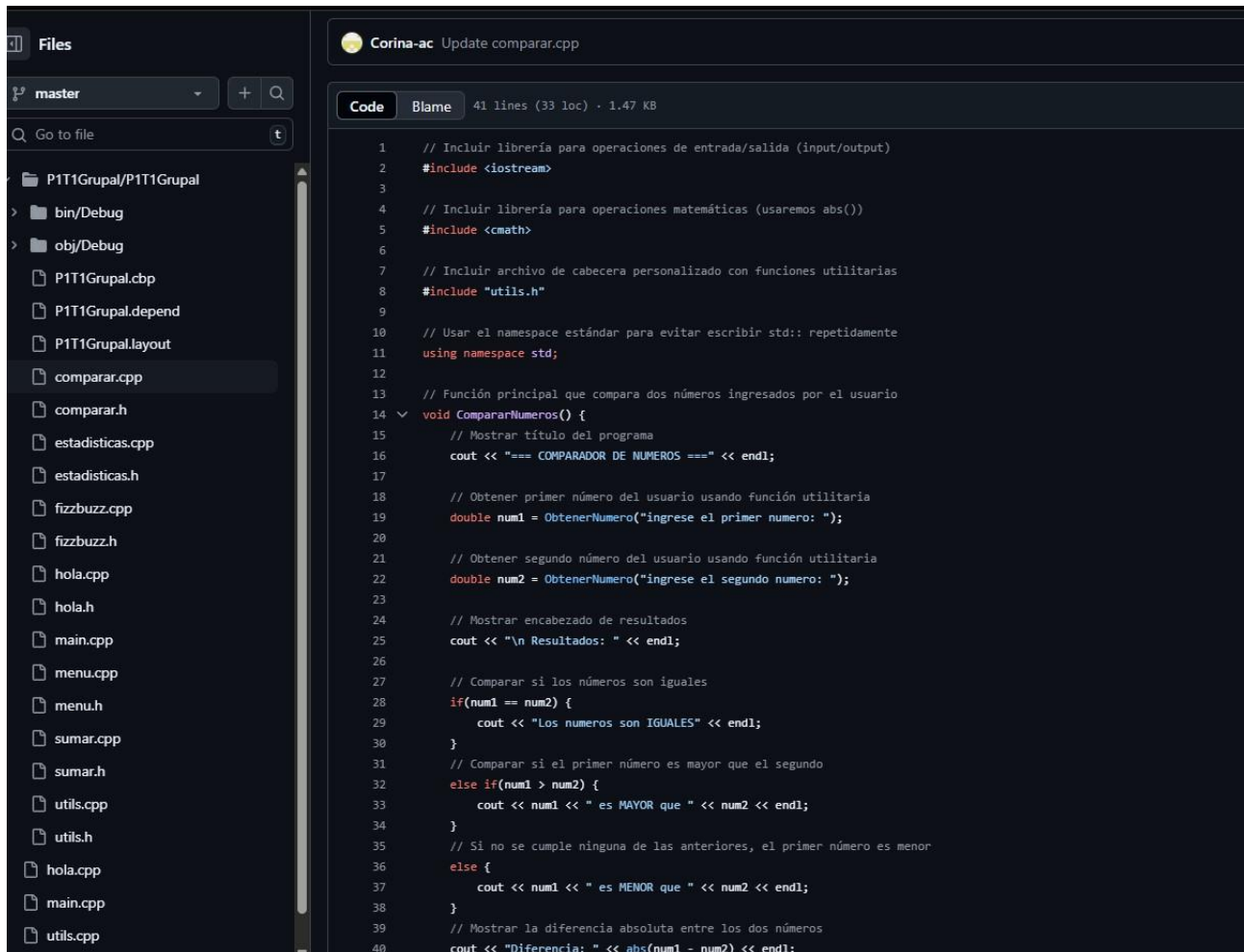
IV. 4. Conclusiones

- La programación constituye una competencia esencial en la era digital, ya que permite la creación de soluciones automatizadas, el análisis eficiente de datos y el desarrollo de tecnologías que impactan múltiples áreas del conocimiento humano.
- Dominar los fundamentos de la programación no solo mejora la capacidad para resolver problemas complejos, sino que también fortalece el pensamiento lógico, la creatividad y la estructuración del pensamiento computacional, habilidades valiosas en cualquier disciplina.
- El lenguaje C++ ofrece un equilibrio entre el control de bajo nivel del hardware y la abstracción de alto nivel, lo que lo convierte en una herramienta poderosa tanto para sistemas embebidos como para aplicaciones orientadas a objetos de gran escala.
- Aprender C++ permite al programador comprender a fondo conceptos clave como la gestión de memoria, la orientación a objetos y la eficiencia algorítmica, haciendo de este lenguaje una excelente base para el desarrollo profesional en ingeniería de software.

V. 5. Recomendaciones

- Fomentar la práctica constante mediante ejercicios de codificación diaria, ya que la programación se domina principalmente a través de la experiencia práctica y la resolución de problemas reales.
- Utilizar entornos de desarrollo simples y didácticos, como Dev-C++ o Code::Blocks, para facilitar la comprensión de los conceptos iniciales sin distraer al estudiante con configuraciones complejas.
- Promover el análisis y la planificación previa al desarrollo del código, utilizando diagramas de flujo o pseudocódigo, con el fin de mejorar la lógica y estructura de los programas antes de su implementación.
- Complementar el aprendizaje con recursos actualizados, como libros, tutoriales interactivos y plataformas en línea, para reforzar los conocimientos teóricos y explorar nuevas herramientas y técnicas de programación.

VI. 6. Anexos:



The image shows a code editor interface. On the left is a file explorer with a tree view showing a project structure. The selected file is 'comparar.cpp'. On the right is the code editor showing the content of 'comparar.cpp'. The code is in C++ and implements a number comparison program. It includes headers for `<iostream>` and `<cmath>`, and a custom header 'utils.h'. The program uses the `std` namespace and defines a function `CompararNumeros()` that prompts the user for two numbers, compares them, and prints the result. It also prints the absolute difference between the two numbers.

```
1 // Incluir librería para operaciones de entrada/salida (input/output)
2 #include <iostream>
3
4 // Incluir librería para operaciones matemáticas (usaremos abs())
5 #include <cmath>
6
7 // Incluir archivo de cabecera personalizado con funciones utilitarias
8 #include "utils.h"
9
10 // Usar el namespace estándar para evitar escribir std:: repetidamente
11 using namespace std;
12
13 // Función principal que compara dos números ingresados por el usuario
14 void CompararNumeros() {
15     // Mostrar título del programa
16     cout << "=== COMPARADOR DE NUMEROS ===" << endl;
17
18     // Obtener primer número del usuario usando función utilitaria
19     double num1 = ObtenerNumero("ingrese el primer numero: ");
20
21     // Obtener segundo número del usuario usando función utilitaria
22     double num2 = ObtenerNumero("ingrese el segundo numero: ");
23
24     // Mostrar encabezado de resultados
25     cout << "\n Resultados: " << endl;
26
27     // Comparar si los números son iguales
28     if(num1 == num2) {
29         cout << "Los numeros son IGUALES" << endl;
30     }
31     // Comparar si el primer número es mayor que el segundo
32     else if(num1 > num2) {
33         cout << num1 << " es MAYOR que " << num2 << endl;
34     }
35     // Si no se cumple ninguna de las anteriores, el primer número es menor
36     else {
37         cout << num1 << " es MENOR que " << num2 << endl;
38     }
39     // Mostrar la diferencia absoluta entre los dos números
40     cout << "Diferencia: " << abs(num1 - num2) << endl;
```

actividad1CPP

Public

forked from jjcruz5/actividad1CPP

master

1 Branch

0 Tags

Go to file

Add file

<> Code

This branch is 9 commits ahead of jjcruz5/actividad1CPP:master .

Corina-ac

Update utils.cpp

P1T1Grupal/P1T1Grupal	Update utils.cpp
hola.cpp	C1 28/04/2025 CJ
main.cpp	C1 28/04/2025 CJ
utils.cpp	C1 28/04/2025 CJ
utils.h	C1 28/04/2025 CJ

yesterday

Local

Codespaces

Clone

HTTPS SSH GitHub CLI

https://github.com/Corina-ac/actividad1CPP.git

Clone using the web URL

Open with GitHub Desktop

Download ZIP

About

No de

Report

Relea

No rele

Create

Packa

Files

master

Go to file

P1T1Grupal/P1T1Grupal

bin/Debug

obj/Debug

P1T1Grupal.cbp

P1T1Grupal.depend

P1T1Grupal.layout

comparar.cpp

comparar.h

estadisticas.cpp

estadisticas.h

fizzbuzz.cpp

fizzbuzz.h

hola.cpp

hola.h

main.cpp

menu.cpp

menu.h

sumar.cpp

sumar.h

utils.cpp

utils.h

hola.cpp

main.cpp

..

comparar.o

Add files via upload

estadisticas.o

Add files via upload

fizzbuzz.o

Add files via upload

hola.o

Add files via upload

main.o

Add files via upload

menu.o

Add files via upload

sumar.o

Add files via upload

utils.o

Add files via upload

VII. 7. Bibliografía/ Referencias

Cáceres Espinoza, L. (2019). *Introducción a la programación: Fundamentos, herramientas y metodologías*. Universidad Nacional de Educación Enrique Guzmán y Valle.

Romero Masís, E., & Rivera Pineda, J. A. (2007). *Introducción a la programación*. Editorial Universidad Don Bosco.

Pérez N., H. O., & Cazarez V., J. L. (2017). *Introducción a la programación*. PUMAEDITORES.